

Frequency-Aware Continual Learning for Smart Contract Vulnerability Detection with Large Language Models

Tenghui Huang, Jiawen Kang, *Senior Member, IEEE*, Dongning Liu, *Senior Member, IEEE*, Changyan Yi, *Senior Member, IEEE*, Chengjun Cai, Anjia Yang, Li Li, and Dong In Kim, *Life Member, IEEE*

Abstract—Smart contract vulnerability detection with Large Language Models (LLMs) faces three causally linked challenges. First, new vulnerability categories demand parameter-efficient adaptation, since full retraining is prohibitive for sequentially arriving tasks. Second, training per-task adapters on a shared backbone causes catastrophic forgetting of previously learned vulnerabilities. Third, the resulting multiplicity of adapters must be consolidated into a single model, since task identity is unknown at inference time. Each challenge arises directly from the solution to its predecessor, making an integrated framework essential. We propose a three-stage pipeline in which each stage addresses one challenge and feeds into the next. The adaptation stage uses Frequency-Aware Low-Rank Adaptation (FA-LoRA), which performs adaptation in the Fourier domain with per-frequency importance gates, requiring only 0.4% trainable parameters while outperforming standard LoRA and QLoRA. The continual learning stage applies Forget-Aware Replay (FAR), which uses these frequency gates to estimate per-sample forgetting risk via loss dynamics and prioritizes vulnerable knowledge for rehearsal, achieving an average Micro-F1 of 0.8022 across sequential tasks. The deployment stage employs Anchor-Protected Progressive Merging (APPM), which exploits the asymmetric generalization produced by FAR training to identify the strongest-generalizing adapter as an anchor and consolidates all adapters into a single model via anchor-protected weighted merging with frequency-domain gate competition. APPM achieves a Micro-F1 of 0.8085, within 2.7% of the independent per-task upper bound, at a merge cost of 156 ms and no additional runtime memory. Extensive experiments on the DIVE benchmark confirm that the proposed framework jointly addresses efficient adaptation, forgetting mitigation, and deployment consolidation for smart contract vulnerability detection in evolving blockchain ecosystems.

Index Terms—Smart contract, vulnerability detection, continual learning, LLM fine-tuning, catastrophic forgetting.

T. Huang and J. Kang are with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mails: 3123000938@mail2.gdut.edu.cn, kavinkang@gdut.edu.cn).

D. Liu is with the School of Computer Science and Technology, Guangdong University of Technology, Guangzhou 510006, China (e-mail: liudn@gdut.edu.cn).

C. Yi is with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China. (e-mail: changyan.yi@nuaa.edu.cn).

C. Cai is with the Department of Computer Science, City University of Hong Kong (Dongguan), Dongguan, Guangdong 518057, China (e-mail: chengjun.cai@cityudg.edu.cn).

Anjia Yang is with the College of Cyber Security, Jinan University, Guangzhou, Guangdong 510632, China (e-mail: anjiayang@gmail.com).

L. Li is with the Guangdong Institute of Science and Technology Information, Guangzhou, Guangdong, China (e-mail: lichil2306@163.com).

Dong In Kim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: dongin@skku.edu).

Corresponding author: Jiawen Kang.

I. INTRODUCTION

Smart contracts govern billions of dollars in decentralized digital assets on modern blockchains, functioning as the execution backbone of these ecosystems and making their security a matter of paramount importance. Vulnerabilities such as reentrancy, integer overflow, and access control flaws continue to cause severe financial losses despite extensive auditing efforts [1]. Because deployed smart contracts are immutable by design, any vulnerability that escapes pre-deployment detection becomes permanently embedded and cannot be readily corrected afterward. Accurate vulnerability detection is therefore a foundational requirement for blockchain security [2].

Large Language Models (LLMs) have shown strong capability in smart contract vulnerability detection by capturing complex program semantics and reasoning over security patterns [3]–[5]. Existing LLM-based approaches, however, assume that all vulnerability categories are known at training time. In practice, smart contract platforms, protocols, and attack techniques continue to evolve, introducing new vulnerability classes over time [6]. A model trained solely on historical data becomes increasingly incomplete as new attack patterns surface, creating a persistent gap between its detection capability and the actual threat landscape. A deployed detector must therefore incorporate newly discovered vulnerabilities while preserving previously acquired knowledge. Static models cannot handle emerging categories, full retraining is prohibitively expensive, and separate task-specific models incur excessive storage and inference overhead [7].

Continual learning (CL) provides a natural solution by enabling a model to learn new tasks sequentially while retaining previously learned knowledge. Applying CL to LLM-based vulnerability detection, however, introduces three challenges that are causally linked. The first challenge is parameter-efficient adaptation. Full fine-tuning is prohibitively expensive for a stream of sequentially arriving tasks, yet per-task adapters offer a practical alternative by sharing a frozen backbone and requiring only a small fraction of parameters to be updated [8], [9]. The second challenge arises directly from this solution. Sequentially training per-task adapters on a shared backbone inevitably overwrites parameters critical to previously learned vulnerabilities, causing catastrophic forgetting [10], [11]. The third challenge follows from the remedy to forgetting. Mitigating forgetting via CL mechanisms produces multiple task-specific adapters, but maintaining separate adapters requires

knowing the task identity at inference time, which is impractical in deployment, while naively merging them causes destructive parameter interference. Each challenge is a direct consequence of solving the one before it, and no single challenge can be resolved in isolation.

To address this causal chain, we propose a three-stage framework that tackles each challenge in sequence. The adaptation stage employs Frequency-Aware Low-Rank Adaptation (FA-LoRA), which learns compact adapters in the Fourier frequency domain with per-frequency importance gates. FA-LoRA requires only 0.4% trainable parameters while outperforming standard LoRA and QLoRA. The continual learning stage uses Forget-Aware Replay (FAR), which applies the frequency gates from FA-LoRA to estimate per-sample forgetting risk via loss dynamics and prioritizes high-risk samples for rehearsal, achieving an average Micro-F1 of 0.8022 across sequential tasks. The deployment stage applies Anchor-Protected Progressive Merging (APPM), which exploits the asymmetric generalization that FAR training produces to identify the strongest adapter as an anchor and consolidate all adapters into a single model via anchor-protected weighted merging and frequency-domain gate competition, achieving a Micro-F1 of 0.8085 within 2.7% of the independent per-task upper bound at a merge cost of only 156 milliseconds.

The three stages share a design dependency. FAR relies on the frequency gates produced by FA-LoRA to estimate which knowledge dimensions are most vulnerable to forgetting. APPM depends on two upstream outputs. The gate parameters from FA-LoRA enable frequency-domain competition during merging, while the asymmetric training outcomes from FAR provide the basis for anchor selection. Since later adapters trained via FAR accumulate knowledge from all previous tasks, they naturally generalize more strongly and serve as stronger anchors. This creates a reinforcing cycle in which better CL training produces better anchors, and better anchors yield better merged models.

The main contributions of this paper are summarized as follows.

- We propose a unified continual learning framework for LLM-based smart contract vulnerability detection that integrates efficient adaptation, forgetting mitigation, and deployment consolidation into a cohesive three-stage pipeline. The framework jointly addresses parameter efficiency, catastrophic forgetting, and multi-task inference, providing an end-to-end solution from sequential task learning to unified model deployment.
- We propose FA-LoRA, a frequency-aware parameter-efficient fine-tuning method that performs low-rank adaptation in the Fourier domain. FA-LoRA requires only 0.4% trainable parameters while consistently outperforming standard LoRA and QLoRA.
- We develop FAR, a continual learning algorithm that prioritizes replay according to per-sample forgetting risk estimated from loss dynamics, achieving the best continual learning performance among compared methods on sequential vulnerability detection tasks.
- We propose APPM, a training-data-free adapter merging strategy that consolidates multiple task-specific adapters

into a single deployment model. APPM achieves performance within 2.7% of the independent per-task upper bound with millisecond-level merging time and no additional runtime memory overhead.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the proposed framework and its architecture. Section IV introduces FA-LoRA. Section V describes the continual learning framework and FAR. Section VI presents APPM. Section VII reports experimental results, and Section VIII concludes the paper.

II. RELATED WORK

A. Smart Contract Vulnerability Detection

Smart contract vulnerability detection has traditionally relied on static analysis and dynamic testing. Static analysis tools such as Slither [12], Securify [13], and SmartCheck [14] detect vulnerabilities through handcrafted rules, data-flow analysis, and formal compliance checking. Fuzzing-based approaches, including ContractFuzzer [15] and Smartian [16], explore execution paths by generating transaction inputs guided by contract interfaces and program dependencies. More recently, SmartAxe [17] targets cross-chain bridge vulnerabilities through fine-grained static analysis, and Satellite [18] detects vulnerabilities caused by subcontract misuse. Although effective for predefined vulnerability patterns, these methods generally suffer from limited semantic understanding and poor generalization to previously unseen attack types [2].

Recent studies have demonstrated the effectiveness of LLMs for smart contract security analysis. GPTScan [3] combines GPT-4 with static analysis to improve vulnerability confirmation, while PropertyGPT [4] incorporates property-guided reasoning to enhance detection coverage. Other studies fine-tune open-source LLMs for vulnerability classification [19]–[22], and recent work further leverages LLMs to augment smart contract decompiler output through semantic recovery [23]. However, these methods assume a static vulnerability taxonomy and lack the ability to adapt to newly emerging vulnerability categories.

B. Parameter-Efficient Fine-Tuning for LLMs

PEFT has become the standard approach for efficient LLM adaptation, with training data transparency emerging as a growing concern [24]. LoRA [8] freezes the backbone and injects trainable low-rank matrices, drastically cutting the parameter count. QLoRA [9] further combines low-rank adaptation with 4-bit quantization for commodity-hardware fine-tuning. More recent variants incorporate sparse adapters [25], [26], adaptive rank allocation, and learnable gating.

More recently, frequency-domain PEFT methods have attracted increasing attention. FourierFT [27] learns parameter updates in the Fourier domain, while FouRA [28] further demonstrates the advantages of frequency-domain representations for multi-task adapter merging. However, existing methods typically employ fixed frequency selection strategies and focus primarily on single-task adaptation. Their applicability to continual learning scenarios remains largely unexplored.

C. Continual Learning for Sequential Tasks

CL aims to enable models to acquire new knowledge without forgetting previously learned tasks [29], [30]. Existing approaches can generally be categorized into replay-based, regularization-based, and architectural methods. Replay-based methods, such as DER [11], mitigate forgetting by revisiting samples from previous tasks. Regularization methods, including EWC [10], Online EWC [31], and MAS [32], preserve important parameters by constraining weight updates. Architectural approaches allocate dedicated model capacity for different tasks to reduce interference [33].

Although continual learning has achieved considerable success in computer vision and natural language processing, its application to smart contract vulnerability detection remains largely unexplored. Related work on federated unlearning [34] and one-shot federated learning [35] addresses knowledge retention in distributed settings, though the sequential multi-label scenario considered in this paper poses distinct challenges. Unlike conventional class-incremental benchmarks, vulnerability detection is inherently multi-label, and vulnerability categories often exhibit strong semantic correlations, making catastrophic forgetting more challenging to address.

D. Model Merging for Multi-Task LLMs

Model merging aims to consolidate multiple task-specific models into a single model without additional training, thereby reducing deployment cost [7]. Representative weight-space approaches include Task Arithmetic [36], TIES-Merging [37], DARE [38], and RegMean [39]. Recent studies further investigate Fisher-weighted merging [40], frequency-domain merging [28], [41], and SVD-based LoRA composition [42].

Most existing merging methods assume that all task adapters contribute equally during merging and therefore ignore the asymmetric knowledge accumulation introduced by continual learning. They are not designed for deployment scenarios where later tasks inherit knowledge acquired from earlier ones.

Existing studies have advanced smart contract vulnerability detection, parameter-efficient fine-tuning, continual learning, and model merging as separate threads. Smart contract detectors assume static vulnerability distributions, PEFT methods prioritize efficient adaptation without supporting continual learning, CL methods focus on mitigating forgetting without considering deployment efficiency, and model merging approaches overlook the sequential knowledge accumulation inherent in continual learning. An integrated framework that jointly addresses adaptation efficiency, forgetting mitigation, and deployment consolidation is still lacking.

III. PROPOSED FRAMEWORK

A. Problem Setting and Design Rationale

In a production deployment scenario, a smart contract vulnerability detector faces a stream of temporally ordered vulnerability tasks $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_K$ with evolving vulnerability patterns. The detector must adapt to each new task with minimal computational overhead, since full retraining of LLMs is prohibitively expensive. At the same time, accuracy

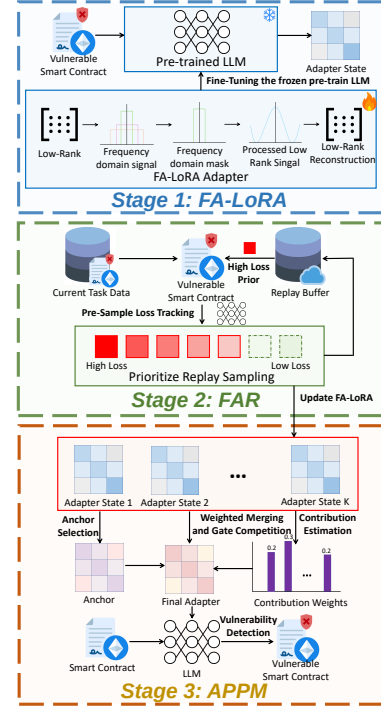


Fig. 1: Overview of the proposed continual learning framework for LLM-based smart contract vulnerability detection.

on previously learned vulnerability categories must be preserved against catastrophic forgetting, and the deployed model must remain a single set of parameters detecting all learned vulnerabilities without task-identity information at inference time. These three requirements pull in opposing directions: mitigating forgetting typically requires storing historical data or parameters, conflicting with efficient adaptation, while maintaining task-specific models violates unified deployment. Our framework resolves this tension through three integrated components, as illustrated in Fig. 1.

B. Three-Stage Pipeline Architecture

The framework operates as a three-stage pipeline.

Stage 1: Efficient Adaptation via FA-LoRA. When a new vulnerability task $\mathcal{T}_k$ arrives, the system fine-tunes only a lightweight FA-LoRA adapter while keeping the pre-trained LLM backbone frozen. The adapter introduces frequency-domain gating and sparse frequency selection, requiring only 0.4% trainable parameters, with each task producing a compact adapter state of negligible storage cost.

Stage 2: Forgetting Mitigation via FAR. During sequential training across tasks, the system maintains a fixed-capacity replay buffer. After each task completes, per-sample forgetting risk is estimated from recent loss dynamics. During subsequent training, samples with higher forgetting risk receive proportionally higher replay probability, ensuring that vulnerable knowledge receives prioritized rehearsal. This loss-dynamics-driven prioritization achieves stronger retention than uniform replay at identical buffer capacity.

Stage 3: Unified Deployment via APPM. After all K tasks have been learned, the K task-specific FA-LoRA adapters are

consolidated into a single unified adapter through APPM. The merging process operates entirely in parameter space without accessing training data. It first identifies the best-generalizing adapter as the anchor, then performs norm-weighted parameter aggregation for LoRA matrices and anchor-boosted frequency-domain competition for gate parameters. The resulting merged adapter supports multi-task inference with accuracy approaching the independent per-task upper bound, while introducing no runtime overhead.

C. Component Interaction

The three components are designed so that each stage produces outputs used by downstream stages. The frequency-domain parameterization of FA-LoRA provides adapters whose spectral structure directly supports two downstream operations. First, the frequency gate parameters indicate which knowledge dimensions are most vulnerable to forgetting, informing the loss-dynamics-driven prioritization in FAR. Second, the same gate parameters enable the frequency-domain competition mechanism of APPM, where each frequency bin is allocated to the task that activates it most strongly. Conventional LoRA adapters lack explicit frequency modeling and therefore cannot support either operation.

In turn, the anchor protection mechanism of APPM exploits the sequential knowledge accumulation inherent to continual learning, where later adapters trained on richer knowledge serve as stronger anchors for the merged representation. This creates a reinforcing cycle in which better CL algorithms produce better anchors, and better anchors yield better merged models. The pipeline maintains a single frozen architecture throughout, stores only compact adapter states, and requires only milliseconds of CPU time for final merging.

IV. FREQUENCY-AWARE LOW-RANK ADAPTATION

A. Design Motivation

Low-Rank Adaptation (LoRA) [8] has emerged as an effective PEFT technique by introducing trainable low-rank updates into frozen large language models. Specifically, given a pre-trained weight matrix $\mathbf{W} \in \mathbb{R}^{d \times d}$, LoRA represents its adaptation as:

$$\Delta \mathbf{W} = \mathbf{U}\mathbf{V}, \quad (1)$$

where $\mathbf{U} \in \mathbb{R}^{d \times r}$ and $\mathbf{V} \in \mathbb{R}^{r \times d}$ with $r \ll d$. The forward propagation is formulated as:

$$\mathbf{h} = \mathbf{x}\mathbf{W} + \alpha \mathbf{x}(\mathbf{U}\mathbf{V}), \quad (2)$$

where α is a scaling factor and $\mathbf{W}$ remains frozen.

Although LoRA significantly reduces trainable parameters, its low-rank update mechanism treats all adaptation components equally. This uniform optimization may limit the capability of LoRA in continual learning scenarios, where different tasks may require different adaptation patterns. In particular, sequential vulnerability detection tasks may contain both general security knowledge and task-specific vulnerability characteristics, making it important to selectively preserve informative adaptation signals.

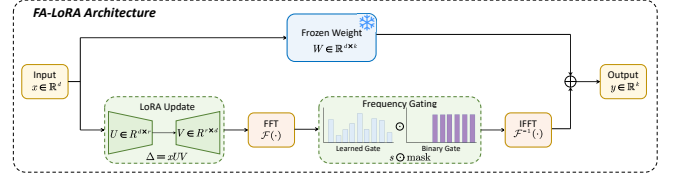


Fig. 2: Architecture of FA-LoRA.

Recent studies on frequency-domain parameter-efficient adaptation [27], [28] suggest that neural representations contain diverse frequency components with different information characteristics. Motivated by this observation, FA-LoRA introduces frequency-domain decomposition into LoRA updates and adaptively controls frequency components through learnable gating and sparse frequency selection.

B. Frequency-Aware Adapter Architecture

The overall architecture of FA-LoRA is illustrated in Fig. 2. Given an input representation $\mathbf{x} \in \mathbb{R}^{n \times d}$, FA-LoRA first generates the standard low-rank adaptation signal:

$$\Delta \mathbf{H} = \mathbf{x}(\mathbf{U}\mathbf{V}), \quad (3)$$

where $\Delta \mathbf{H}$ denotes the LoRA-induced representation update. Instead of directly applying the low-rank update in the original feature space, FA-LoRA transforms it into the frequency domain using the discrete DFT:

$$\mathbf{F}_{\Delta \mathbf{H}} = \mathcal{F}(\Delta \mathbf{H}), \quad (4)$$

where $\mathcal{F}(\cdot)$ represents the Fourier transform operation. The frequency representation $\mathbf{F}_{\Delta \mathbf{H}}$ explicitly separates the adaptation signal into different frequency components.

To adaptively adjust the contribution of different frequency components, FA-LoRA introduces a learnable frequency gate:

$$\mathbf{s} = \sigma(\mathbf{g}), \quad (5)$$

where $\mathbf{g}$ denotes trainable frequency importance parameters and $\sigma(\cdot)$ is the sigmoid activation function. The obtained gate vector $\mathbf{s} \in [0, 1]^r$ provides soft frequency-wise modulation.

Furthermore, FA-LoRA applies sparse frequency selection to focus adapter capacity on the most informative components. A binary frequency mask is defined as:

$$\text{mask}_i = \begin{cases} 1, & i \in \mathcal{K}, \\ 0, & \text{otherwise}, \end{cases} \quad (6)$$

where $\mathcal{K}$ represents the selected frequency indices:

$$\mathcal{K} = \text{TopK}(|\mathbf{s}|, \lceil \gamma r \rceil). \quad (7)$$

Here, γ controls the retained frequency ratio. Following previous observations that high-frequency components often contain fine-grained adaptation signals beneficial for task-specific knowledge adjustment, we retain only the most informative high-frequency components.

The gated frequency representation is calculated as:

Algorithm 1: FA-LoRA Forward Pass (Single Layer)

```

1 Initialize frozen weight  $\mathbf{W} \in \mathbb{R}^{d \times d}$ .
2 Initialize low-rank parameters  $\mathbf{U} \in \mathbb{R}^{d \times r}$ ,  $\mathbf{V} \in \mathbb{R}^{r \times d}$ .
3 Initialize frequency gate  $\mathbf{g} \in \mathbb{R}^r$  and retention ratio  $\gamma$ .
4 for each input  $\mathbf{x} \in \mathbb{R}^{n \times d}$  do
5   ### Frozen Projection ###
6   Compute the base output:  $\mathbf{z} \leftarrow \mathbf{x}\mathbf{W}$ .
7   ### Low-Rank Adapter ###
8   Compute the low-rank update:  $\Delta\mathbf{H} \leftarrow \mathbf{x}(\mathbf{U}\mathbf{V})$ .
9   ### Frequency-Domain Gating ###
10  Transform to frequency domain:  $\mathbf{F}_{\Delta\mathbf{H}} \leftarrow \mathcal{F}(\Delta\mathbf{H})$ .
11  Compute gate activation:  $\mathbf{s} \leftarrow \sigma(\mathbf{g})$ .
12  Select top- $k$  frequencies:  $\mathcal{K} \leftarrow \text{TopK}(|\mathbf{s}|, \lceil \gamma r \rceil)$ .
13  Construct binary mask:  $\text{mask} \leftarrow \mathbb{K}[i \in \mathcal{K}]$ .
14  Apply gated sparse mask:
     $\mathbf{F}_{FA} \leftarrow \mathbf{F}_{\Delta\mathbf{H}} \odot (\mathbf{s} \odot \text{mask})$ .
15  ### Inverse Transform ###
16  Reconstruct adapter output:  $\Delta\mathbf{H}_{FA} \leftarrow \mathcal{F}^{-1}(\mathbf{F}_{FA})$ .
17  ### Final Output ###
18  Combine outputs:  $\mathbf{h} \leftarrow \mathbf{z} + \alpha\Delta\mathbf{H}_{FA}$ .
19 end

```

$$\mathbf{F}_{FA} = \mathbf{F}_{\Delta\mathbf{H}} \odot (\mathbf{s} \odot \text{mask}), \quad (8)$$

where $\odot$ is the element-wise product. The final FA-LoRA update is reconstructed via inverse FFT:

$$\Delta\mathbf{H}_{FA} = \mathcal{F}^{-1}(\mathbf{F}_{FA}), \quad (9)$$

where $\mathcal{F}^{-1}(\cdot)$ denotes the inverse Fourier transform. Therefore, the complete forward computation of a FA-LoRA-enhanced transformer layer is:

$$\mathbf{h} = \mathbf{x}\mathbf{W} + \alpha\Delta\mathbf{H}_{FA}. \quad (10)$$

FA-LoRA is applied to the query and value projection matrices $\mathbf{W}_q$ and $\mathbf{W}_v$ of each transformer layer, following the conventional LoRA configuration. Algorithm 1 summarizes the forward propagation procedure of FA-LoRA.

C. Parameter Efficiency

FA-LoRA introduces only the low-rank matrices and frequency gate parameters while keeping the original LLM parameters frozen. For each adapted linear layer, the number of trainable parameters is:

$$N_{\text{FA-LoRA}} = 2dr + r, \quad (11)$$

where r is the LoRA rank and d is the hidden dimension.

Compared with full fine-tuning requiring d^2 trainable parameters for each linear layer, FA-LoRA reduces the optimization space from quadratic to linear complexity with respect to the hidden dimension, requiring only a small fraction of the total model parameters to be trainable.

This substantial parameter reduction enables efficient continual fine-tuning while preserving the expressive capability of large language models.

D. Training Objective

For multi-label smart contract vulnerability detection with L vulnerability categories, FA-LoRA is optimized using the binary cross-entropy objective:

$$\mathcal{L} = -\frac{1}{L} \sum_{i=1}^L [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)], \quad (12)$$

where $y_i \in \{0, 1\}$ is the ground-truth label and $\hat{y}_i$ is the predicted probability for vulnerability category i . During training, only $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{g}$ are updated while all pre-trained LLM parameters remain frozen.

Before evaluating FA-LoRA in continual learning scenarios, we first validate its effectiveness as an independent PEFT method. Section VII compares FA-LoRA with representative PEFT baselines, including LoRA [8], QLoRA [9], SLoRA [22], FourierFT [27], and WaRA [43], on the complete DIVE dataset.

V. CONTINUAL LEARNING FOR SEQUENTIAL VULNERABILITY TASKS

A. Problem Formulation

In real-world auditing, vulnerability knowledge evolves over time. A vulnerability category can only be labeled after its public discovery, so a deployed model should only access patterns known before that time point. This temporal constraint leads directly to a continual learning formulation.

Given a chronologically ordered contract corpus, we sort all contracts according to their deployment or commit timestamps and divide them into K consecutive time intervals:

$$\mathcal{I}_k = [t_{k-1}, t_k), \quad k = 1, \dots, K. \quad (13)$$

Each interval defines a continual learning task:

$$\mathcal{T}_k = \mathcal{D}_k = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N_k}, \quad (14)$$

where $\mathbf{x}_i$ denotes a smart contract representation and $\mathbf{y}_i \in \{0, 1\}^L$ is the multi-label vulnerability annotation over L vulnerability categories.

Under the temporal partition, early tasks contain vulnerability patterns known at earlier stages, while later tasks introduce newly emerging vulnerability characteristics. The concrete benchmark construction adopted in our evaluation is detailed in Section VII.

The objective of continual learning is to sequentially optimize the model over tasks:

$$\mathcal{T}_1 \rightarrow \mathcal{T}_2 \rightarrow \dots \rightarrow \mathcal{T}_K, \quad (15)$$

while preserving performance on previously learned tasks. Let $\text{F1}_{i,j}$ denote the performance on task j after learning task i , and let FWT and BWT denote Forward Transfer and Backward Transfer, respectively:

$$\text{FWT} = \frac{1}{K-1} \sum_{k=2}^K (\text{F1}_{k-1,k} - \text{F1}_{\text{base},k}), \quad (16)$$

$$\text{BWT} = \frac{1}{K-1} \sum_{k=2}^K \frac{1}{k-1} \sum_{j=1}^{k-1} (\text{F1}_{k,j} - \text{F1}_{j,j}). \quad (17)$$

Positive FWT indicates beneficial knowledge transfer to future tasks, whereas negative BWT reflects performance degradation caused by catastrophic forgetting.

B. Estimating Forgetting from Loss Dynamics

While replay-based methods alleviate forgetting, they treat all stored samples uniformly. Yet forgetting is highly sample-dependent. Some samples stay robust across tasks, while others degrade sharply. Uniform replay thus wastes limited buffer capacity on knowledge already retained.

To this end, FAR estimates per-sample forgetting risk from recent training losses. Each buffer entry stores the input $\mathbf{x}_i$, the vulnerability label $\mathbf{y}_i$, and a forgetting indicator ℓ_i given by the sample-wise binary cross-entropy loss:

$$\ell_i = -\frac{1}{L} \sum_{m=1}^L [y_{i,m} \log(\hat{y}_{i,m}) + (1 - y_{i,m}) \log(1 - \hat{y}_{i,m})], \quad (18)$$

where $y_{i,m}$ is the ground-truth label for category m and $\hat{y}_{i,m}$ is the current model prediction. After each task $\mathcal{T}_k$, all buffered samples are re-evaluated through an inference pass and their loss values updated via Eq. (18). These losses are used only to estimate forgetting risk and assign replay probabilities, without adding any optimization objective.

C. Forget-Aware Replay Sampling

During replay, samples are selected according to a temperature-scaled probability distribution:

$$p_i = \frac{\exp(\ell_i/\tau)}{\sum_{j=1}^{|\mathcal{M}|} \exp(\ell_j/\tau)}, \quad (19)$$

where τ controls the prioritization intensity. A smaller τ concentrates replay probability on high-loss samples, while a larger τ flattens the distribution toward uniform sampling.

The proposed strategy has two advantages. First, it is task-agnostic because the sampling criterion depends only on prediction difficulty rather than task identity. Second, it is self-adaptive because successfully rehearsed samples receive lower future probabilities.

Algorithm 2 summarizes the sampling procedure. Fig. 3 illustrates the overall workflow of FAR.

D. Overall Training Objective

The complete FAR procedure can be formalized as follows. When task $\mathcal{T}_k$ arrives, the trainable FA-LoRA parameters Θ are optimized against a combined objective that merges the current-task loss with a rehearsal loss over the prioritized replay batch:

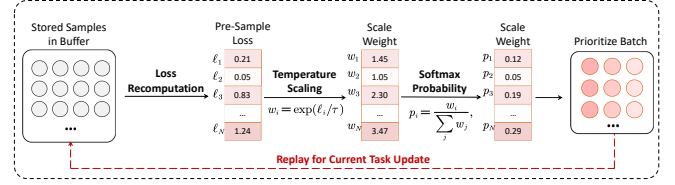


Fig. 3: The FAR mechanism performing forget-aware prioritization via per-sample loss recomputation and temperature-scaled softmax sampling.

Algorithm 2: Forget-Aware Replay Sampling

- 1 Initialize replay buffer $\mathcal{M} = \{(\mathbf{x}_i, \mathbf{y}_i, \ell_i)\}_{i=1}^{|\mathcal{M}|}$.
- 2 Set temperature τ and replay batch size B_r .
- 3 **### Compute Selection Weights ###**
- 4 **for each sample i in $\mathcal{M}$ do**
- 5 | Compute loss-based weight: $w_i \leftarrow \exp(\ell_i/\tau)$.
- 6 **end**
- 7 **### Prioritized Sampling ###**
- 8 Normalize to softmax probabilities: $p_i \leftarrow w_i / \sum_j w_j$.
- 9 Draw B_r samples from $\mathcal{M}$ with probabilities $\{p_i\}$ to form replay batch $\mathcal{B}_r$.
- 10 **return $\mathcal{B}_r$**

$$\mathcal{L}_{\text{CL}}^{(k)}(\Theta) = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{T}_k} \ell(\mathbf{x}_i, \mathbf{y}_i; \Theta) + \lambda \frac{1}{|\mathcal{B}_r|} \sum_{(\mathbf{x}_j, \mathbf{y}_j) \in \mathcal{B}_r} \ell(\mathbf{x}_j, \mathbf{y}_j; \Theta), \quad (20)$$

where $\mathcal{B}_r \subseteq \mathcal{M}$ is the replay batch drawn with the probabilities $\{p_i\}$ of Eq. (19), and λ balances adaptation to the new task against retention of previously learned knowledge, with its value determined by the replay batch ratio in Table II.

The forgetting risk that drives the prioritization in Eq. (19) is quantified by the change of the prediction loss of each buffered sample across consecutive task boundaries. After task $\mathcal{T}_k$ is learned, the forgetting risk of sample i is defined as

$$\Delta \ell_i^{(k)} = \ell_i^{(k)} - \ell_i^{(k-1)}, \quad (21)$$

where $\ell_i^{(k)}$ is the per-sample loss of Eq. (18) recomputed after training on $\mathcal{T}_k$, and the loss recorded upon the entry of the sample into the buffer serves as its baseline. A large positive $\Delta \ell_i^{(k)}$ indicates that the knowledge carried by sample i is being actively overwritten, so the sample receives a higher replay probability in subsequent tasks.

Finally, the temperature-scaled distribution of Eq. (19) admits a principled interpretation: it is the closed-form solution of an entropy-regularized maximization of the expected rehearsal loss,

$$\{p_i\} = \arg \max_{\{p_i\}} \left[\sum_{i=1}^{|\mathcal{M}|} p_i \ell_i - \tau \sum_{i=1}^{|\mathcal{M}|} p_i \log p_i \right], \quad (22)$$

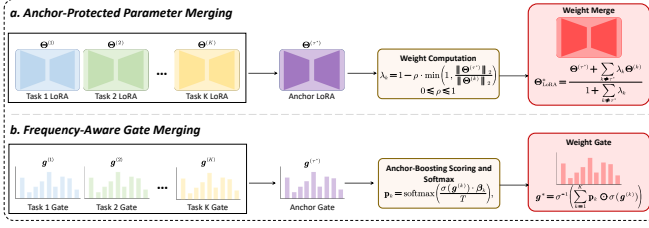


Fig. 4: The two-stage APPM workflow.

subject to $\sum_{i=1}^{|\mathcal{M}|} p_i = 1$. This concentrates rehearsal on high-risk samples, while the entropy term controlled by τ prevents the distribution from collapsing onto a single sample. Taken together, Eqs. (20)–(22) fully specify FAR, in which forgetting risk is estimated from loss dynamics, converted into sampling probabilities, and injected into the training objective through prioritized rehearsal.

VI. ANCHOR-PROTECTED PROGRESSIVE MERGING FOR UNIFIED INFERENCE

A. Problem Definition

Assume that continual learning produces K task-specific FA-LoRA adapters

$$\mathcal{A} = \{\Theta^{(1)}, \Theta^{(2)}, \dots, \Theta^{(K)}\}, \quad (23)$$

where

$$\Theta^{(k)} = \{(U_l^{(k)}, V_l^{(k)}, g_l^{(k)})\}_{l=1}^L \quad (24)$$

denotes the FA-LoRA parameters of task k spanning L transformer layers. APPM produces a single unified adapter

$$\Theta^* = \Phi(\Theta^{(1)}, \dots, \Theta^{(K)}), \quad (25)$$

where $\Phi(\cdot)$ denotes the merging operator. The merging process is performed entirely in parameter space without accessing any training samples, making APPM a post-training data-free adapter merging method.

Figure 4 illustrates the overall workflow of APPM. APPM first identifies an anchor adapter, then performs anchor-protected weighted merging of LoRA matrices, and finally consolidates the frequency gates via softmax competition in the frequency domain.

B. Anchor-Protected Parameter Merging

A central challenge in adapter merging is that different tasks push shared parameters in opposing directions, so naive averaging produces destructive interference. Rather than treating all task adapters equally, APPM first selects an anchor adapter serving as the reference during merging, since later stages accumulate more knowledge and exhibit stronger representation capability. APPM therefore estimates the importance of each adapter by its aggregated parameter norm:

$$N_k = \sum_{l=1}^L \left(\|U_l^{(k)}\|_F^2 + \|V_l^{(k)}\|_F^2 + \|g_l^{(k)}\|_2^2 \right). \quad (26)$$

The anchor adapter is selected as

$$\tau^* = \arg \max_k N_k. \quad (27)$$

Once the anchor is identified, APPM aggregates parameters via weighted averaging, where each non-anchor adapter contributes in proportion to its relative parameter magnitude:

$$\lambda_k = 1 - \rho \cdot \min \left(1, \frac{\|\Theta^{(\tau^*)}\|_2}{\|\Theta^{(k)}\|_2} \right), \quad (28)$$

where ρ controls the strength of anchor protection.

The merged LoRA parameters are computed as

$$\Theta_{\text{LoRA}}^* = \frac{\Theta^{(\tau^*)} + \sum_{k \neq \tau^*} \lambda_k \Theta^{(k)}}{1 + \sum_{k \neq \tau^*} \lambda_k}. \quad (29)$$

This formulation preserves the dominant knowledge of the anchor while incorporating complementary information from other tasks into the final merged adapter.

C. Frequency-Domain Gate Competition

Unlike the LoRA matrices, the gate parameters in FA-LoRA determine the activation of frequency components and therefore directly influence the frequency characteristics of the learned adapter. Simple parameter averaging may suppress important task-specific frequency responses.

To preserve informative frequency components, APPM formulates gate merging as a frequency-wise competition among task adapters. Specifically, the gate activations are first transformed into probability space through the sigmoid function. Each task then competes for every frequency component using a temperature-controlled softmax:

$$\mathbf{p}_k = \text{softmax} \left(\frac{\sigma(g^{(k)}) \cdot \beta_k}{T} \right), \quad (30)$$

where β_k denotes the anchor-aware importance coefficient and T controls the sharpness of the competition.

The merged gate parameters are obtained by

$$g^* = \sigma^{-1} \left(\sum_{k=1}^K \mathbf{p}_k \odot \sigma(g^{(k)}) \right), \quad (31)$$

where $\sigma^{-1}(\cdot)$ denotes the inverse sigmoid function.

This competition mechanism assigns each frequency component to the task that contributes the strongest activation while preserving the dominant role of the anchor adapter, thereby reducing frequency-domain interference among different tasks.

D. Computational Complexity

APPM is performed only once after continual learning has finished and therefore introduces no additional training cost. The computational complexity is dominated by parameter aggregation across all adapters:

$$\mathcal{O}(KL(dr + r)), \quad (32)$$

Algorithm 3: Anchor-Protected Progressive Merging

```

1 Initialize  $K$  task adapters  $\{\Theta^{(1)}, \dots, \Theta^{(K)}\}$ .
2 Set protection coefficient  $\rho$ , anchor coefficient  $\beta$ , and
  temperature  $T$ .
3 ### Anchor Identification ###
4 for each task  $k = 1, \dots, K$  do
5   Compute parameter norm:
   
$$N_k \leftarrow \sum_l (\|U_l^{(k)}\|_F^2 + \|V_l^{(k)}\|_F^2 + \|g_l^{(k)}\|_2^2).$$

6 end
7 Select anchor:  $\tau^* \leftarrow \arg \max_k N_k$ .
8 ### Weighted LoRA Merging ###
9 for each non-anchor task  $k \neq \tau^*$  do
10  Compute contribution weight:
   
$$\lambda_k \leftarrow 1 - \rho \cdot \min(1, \|\Theta^{(\tau^*)}\|_2 / \|\Theta^{(k)}\|_2).$$

11 end
12 Merge LoRA parameters via weighted averaging using
    $\{\lambda_k\}$ .
13 ### Frequency Gate Competition ###
14 Compute per-frequency softmax allocation  $\mathbf{p}_k$  using
   (30).
15 Merge frequency gates using (31).
16 ### Final Assembly ###
17 Construct merged adapter:  $\Theta^* \leftarrow \{\Theta_{\text{LoRA}}^*, \mathbf{g}^*\}$ .
18 return  $\Theta^*$ 

```

where K is the number of continual learning tasks, L is the number of adapter-augmented transformer layers, d denotes the hidden dimension, and r is the LoRA rank.

Since APPM operates entirely in parameter space without requiring gradient computation or training data, it introduces negligible deployment overhead. Algorithm 3 summarizes the complete merging procedure.

VII. EXPERIMENTAL EVALUATION

A. Experimental Setup

We implement all experiments on a server with two NVIDIA RTX A6000 GPUs, each with 48 GB of memory, and one RTX 4090 with 24 GB, using Python 3.10 with PyTorch 2.7.1. The base model is LLaMA-3.2-1B [44] with 4-bit NF4 quantization [9]. Detailed hyperparameter settings are provided in Table II.

Dataset and Temporal Task Construction. We use the DIVE benchmark, which contains 22,330 real-world smart contracts with 8 multi-label vulnerability categories [45]. To construct a deployment-realistic CL evaluation, we sort all contracts by their commit or deployment dates and partition them into $K = 4$ contiguous time intervals, yielding tasks task_A through task_D, consistent with the temporal constraint of Section V-A. As shown in Table I, the four tasks have comparable sizes but shifting vulnerability category distributions. Input features are serialized as structured text with a maximum of 1,024 tokens.

Training. All CL methods train sequentially on task_A through task_D using the AdamW optimizer [46]. Each task produces a compact FA-LoRA adapter state. The complete hyperparameter configuration is summarized in Table II.

TABLE I: DIVE benchmark statistics.

Split	task_A	task_B	task_C	task_D
Train	5,262	5,262	5,262	5,262
Validation	542	504	536	651
Test	530	542	513	648
Total	6,334	6,308	6,311	6,561

TABLE II: Hyperparameter Configuration

Hyperparameter	Value
FA-LoRA rank r [8]	16
Frequency mode [28]	High-frequency retention
Retain fraction γ [28]	0.2
Optimizer [46]	AdamW
Learning rate (CL) [8]	5×10^{-5}
Learning rate (PEFT) [8]	3×10^{-5}
Batch size B [8]	8
Max sequence length	1,024
Epochs per task (CL)	3–5
Epochs (PEFT)	3
Replay buffer capacity [11]	2,000
Replay batch ratio [11]	0.25
FAR temperature τ	2.0
APPM protection ρ	1.0

Evaluation Metrics. We report per-task and average Micro-F1, FWT and BWT as defined in Section V, merge time in milliseconds, CPU memory delta and GPU peak inference memory in MB, and parameter count. We additionally report the Independent per-task upper bound from separate per-task FA-LoRA training, and the No-Adapter lower bound of an untrained model.

B. Parameter Efficiency

Table III summarizes the parameter efficiency of FA-LoRA. With only 5.2M trainable parameters, FA-LoRA achieves strong detection accuracy while requiring approximately 10 MB of storage per task adapter. The frozen base model accounts for 99.6% of parameters, shared across all tasks.

C. Comparison with PEFT Baselines

Before evaluating continual learning performance, we first validate the effectiveness of FA-LoRA as a parameter-efficient fine-tuning method by comparing it against state-of-the-art PEFT approaches on the full DIVE dataset *without* task splitting. This experiment uses the complete 22,330-contract dataset with all 8 vulnerability labels, evaluating single-task detection capability in isolation from CL concerns. All methods share identical optimization settings as detailed in Table II. We evaluate on both LLaMA-3.2-1B and LLaMA-3.2-3B to assess scaling behavior.

TABLE III: Parameter breakdown of FA-LoRA.

Component	Params (M)	Fraction
Total	1,241.0	100.0%
Frozen (LLaMA base)	1,235.8	99.6%
Trainable (FA-LoRA)	5.2	0.4%
— LoRA (U, V)	5.0	0.40%
— Gates (g)	0.2	0.02%
Per-task storage	~10 MB	

We compare seven methods spanning the PEFT design space, including standard LoRA [8], QLoRA [9], SLoRA [22], FourierFT [27], FouRA [28], and WaRA [43], as well as our proposed FA-LoRA. FourierFT and FouRA are two frequency-domain baselines that share the same architecture and differ only in which frequency band is preserved, providing a controlled ablation that isolates the effect of frequency band selection. WaRA performs low-rank adaptation in the wavelet domain. Detailed configurations are listed in Table II.

Table IV reports Micro-F1, Macro-F1, and subset accuracy for all methods on both model scales, alongside trainable parameter counts and on-disk adapter storage size. FA-LoRA is our proposed method, while FouRA and FourierFT serve as frequency-domain baselines that together form a controlled ablation of frequency band selection.

Table IV reports the comparison at both model scales. FA-LoRA attains accuracy close to the strongest baseline while training only 0.4% of the parameters, and at the 3B scale it outperforms both LoRA and QLoRA across all three metrics using 23% fewer parameters than LoRA. This advantage stems from retaining the most informative high-frequency components, which concentrates adapter capacity on vulnerability-relevant patterns. The FouRA versus FourierFT comparison confirms the effect, since FouRA preserves high frequencies and outperforms FourierFT at both scales.

D. Comparison of Continual Learning Algorithms

Fig. 5 presents the comprehensive comparison of CL algorithms on the DIVE benchmark, reporting per-task Micro-F1 after all four tasks have been learned. The Independent upper bound is shown as dashed reference lines.

Fig. 5 reveals several key findings:

As shown in Fig. 5, FAR achieves the best overall CL performance with an average Micro-F1 of 0.8022, outperforming all five baselines and trailing the Independent upper bound by only 3.5%. The loss-dynamics-driven prioritization ensures that the most vulnerable knowledge receives the most rehearsal, yielding stronger retention than uniform replay with identical buffer capacity.

E. Merging Performance

Fig. 6 presents APPM against baseline merging methods. All methods merge the four task-specific FA-LoRA adapters

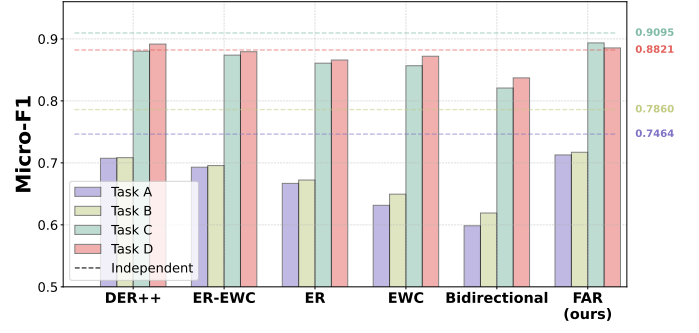


Fig. 5: Continual learning algorithm comparison on the DIVE benchmark. Grouped bars show per-task Micro-F1 for each method, with dashed lines indicating the Independent single-task upper bound.

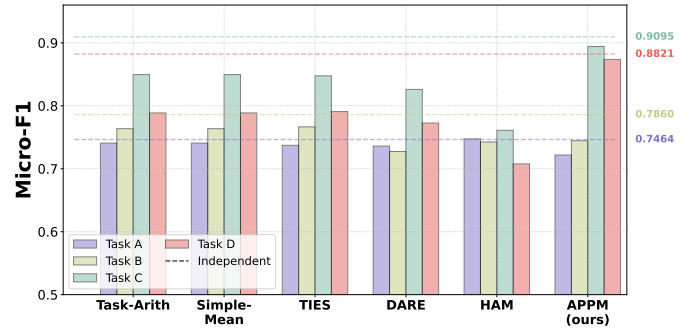


Fig. 6: APPM merging comparison on the DIVE benchmark. Grouped bars show per-task Micro-F1 for each merging method, with dashed lines indicating the Independent single-task upper bound. APPM achieves the best average Micro-F1 of 0.8085, closest to the Independent upper bound.

into a single adapter without accessing training data. Dashed lines mark the Independent upper bound, while dotted lines mark Sequential. Table V provides detailed resource metrics.

As shown in Fig. 6 and Table V, APPM achieves near-optimal multi-task inference with an average Micro-F1 of 0.8085, within 2.7% of the Independent upper bound. On task_C and task_D, APPM outperforms all baseline merging methods, which shows that anchor protection preserves the strong generalization of later adapters. APPM completes merging in 156 ms with zero additional runtime memory overhead.

F. Ablation Studies

1) *APPM Component Ablation*: We ablate the two key mechanisms of APPM: anchor protection for LoRA parameters and frequency-domain gate competition. Fig. 7 reports results with all four component combinations. Each variant merges the same four FAR adapters, where Anchor indicates whether task_D, the identified anchor, is protected and Freq indicates whether frequency-domain softmax competition is used. The Anchor-only variant corresponds to norm-weighted parameter averaging, and the Freq-only variant corresponds to SAM-Orthogonal merging.

TABLE IV: Comparison of PEFT methods on the full DIVE dataset without task splitting.

Method	Quant.	Trainable (M)	Storage (MB)	Micro-F1		Macro-F1		Subset Acc.	
				1B	3B	1B	3B	1B	3B
WaRA [43]	✓	35.77	136.5	0.8398	0.8515	0.6529	0.7133	0.5544	0.5840
QLoRA [9]	✓	1.72	6.6	<u>0.8185</u>	0.8365	<u>0.6133</u>	0.6407	<u>0.5181</u>	0.5298
SLoRA [22]	✓	6.85	26.2	0.8138	0.8305	0.5945	0.6263	0.4966	0.5262
LoRA [8]	bf16	3.42	13.0	0.8094	0.8370	0.5613	0.6356	0.4845	0.5428
FourierFT [27]	✓	0.16	0.6	0.7449	0.7888	0.4724	0.5585	0.3471	0.4331
FouRA [28]	✓	0.55	2.1	0.7635	0.8020	0.4998	0.5930	0.4004	0.4648
FA-LoRA	✓	2.62	10.0	<u>0.8185</u>	<u>0.8424</u>	0.5904	<u>0.6616</u>	0.5074	<u>0.5544</u>

TABLE V: Resource metrics for adapter merging methods. All methods produce identical model architectures.

Method	Merge (ms)	CPU Δ (MB)	Speedup	Δ Ind
Simple-Mean	102	0.0	70×	+5.5%
TIES [37]	2,123	0.0	3.4×	+5.5%
DARE [38]	4,204	0.0	1.7×	+7.9%
HAM g=2	7,164	68.1	1.0×	+11.0%
SFA a=0.5	72	0.0	100×	+33.0%
APPM (ours)	156	0.0	46×	+2.7%

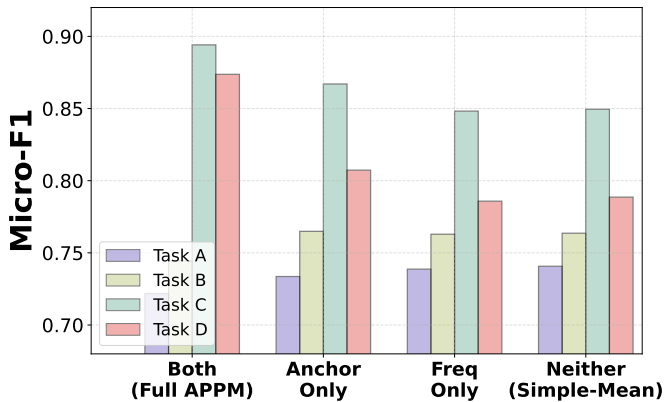


Fig. 7: APPM component ablation. All variants merge 4 FAR adapters. The full APPM configuration achieves the best balance across all four tasks, while removing either anchor protection or frequency competition degrades performance, so both mechanisms are necessary.

G. Sensitivity Analysis

Removing anchor protection causes the largest drop on later tasks, where task_D falls sharply from 0.8737 to 0.8073. Protecting the generalization of the anchor is therefore the primary mechanism. Removing frequency competition instead hurts earlier tasks, where task_A rises from 0.7218 to 0.7387 as the anchor-free variant over-favors early tasks at the cost of later ones. Removing both mechanisms defaults to Simple-Mean, which occupies a middle ground but sacrifices 2.8% overall.

TABLE VI: Backward evaluation matrix for FAR. Bold entries denote in-task F1, entries below the diagonal denote backward transfer, and entries above the diagonal denote forward transfer.

After training	task_A	task_B	task_C	task_D
task_A	0.7495	0.6121	0.6331	0.6028
task_B	0.7253	0.7503	0.7303	0.7065
task_C	0.7162	0.7226	0.8974	0.8864
task_D	0.7128	0.7171	0.8935	0.8854
Forgetting	−0.0367	−0.0332	−0.0039	—

Only the full APPM configuration successfully balances early-task retention with late-task generalization.

1) *Backward Evaluation Matrix*: Table VI reports the stage-wise evaluation matrix for FAR. Each row gives the performance on all four tasks after training up to that stage. In-task performance appears on the diagonal, forgetting appears below it, and forward transfer appears above it.

The matrix reveals that forgetting is concentrated in the earliest task. task_A drops from 0.7495 to 0.7128, a 4.9% relative decline, whereas task_C drops by only 0.4%. This pattern is characteristic of replay-based CL, as the fixed-capacity buffer progressively undersamples earlier tasks when more tasks are added.

We evaluate the sensitivity of four key hyperparameters: FA-LoRA rank r , retention ratio γ , APPM protection strength ρ , and FAR temperature τ . A total of 27 experiments are run, each measured by average Micro-F1 across all four tasks. Per-task results are shown in Fig. 8.

FA-LoRA exhibits strong robustness to rank r , with a performance difference of only 0.0041 between $r=4$ and $r=32$. Frequency-domain gating therefore effectively compensates for reduced rank. Similarly, the frequency retention ratio γ shows negligible sensitivity, where even $\gamma=0.05$ achieves 0.7617 average Micro-F1, nearly matching $\gamma=0.60$ at 0.7627. High-frequency components therefore carry the dominant information. The FAR temperature τ is also effectively a no-op parameter, as all τ values span only 0.0021 average Micro-F1.

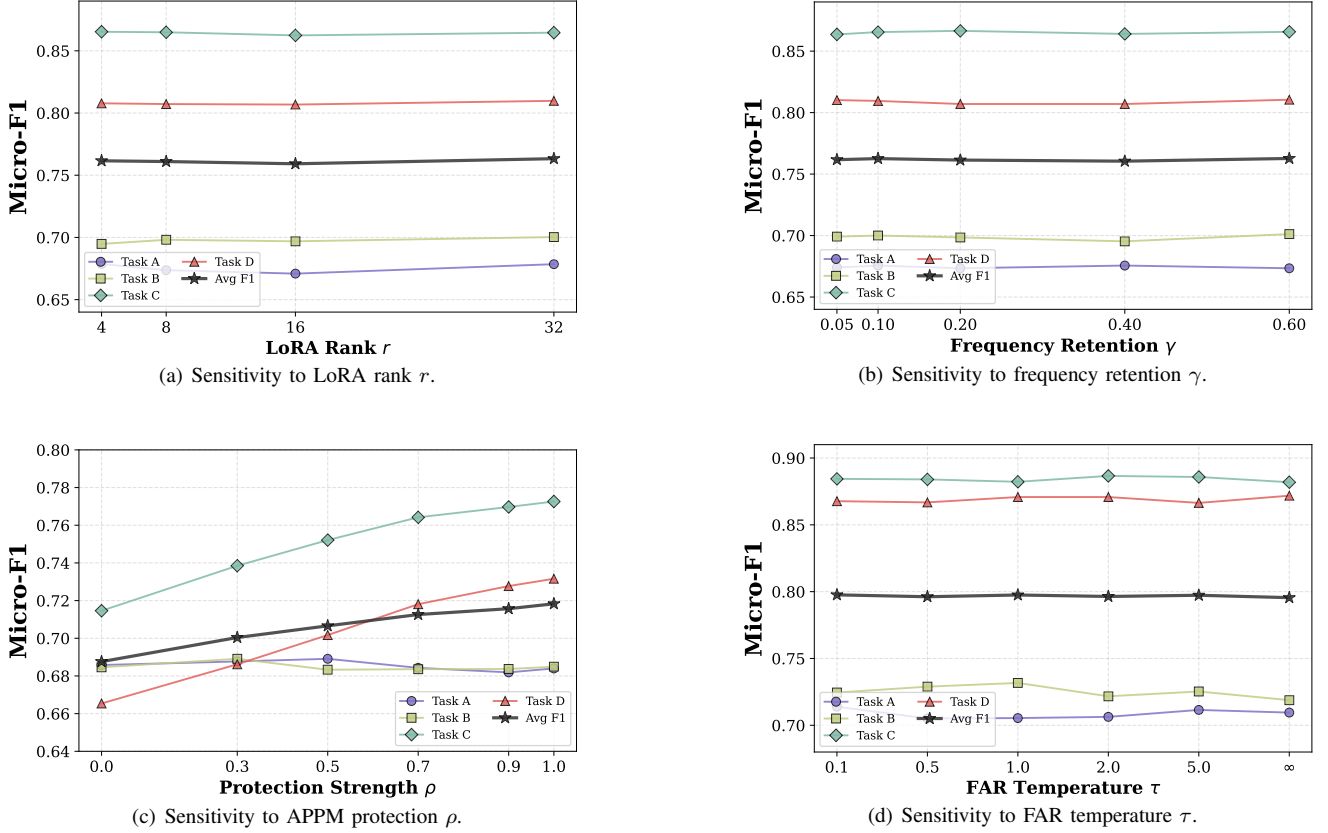


Fig. 8: Sensitivity analysis across four hyperparameter dimensions. (a) LoRA rank r is robust with $\Delta F1=0.0041$. (b) Frequency retention γ is similarly insensitive with $\Delta F1=0.0022$. (c) APPM protection ρ is the only impactful parameter, yielding $+3.07\%$ monotonic improvement. (d) FAR temperature τ has negligible impact with $\Delta F1=0.0021$.

In contrast, the APPM protection strength ρ is the most impactful hyperparameter. Setting $\rho=1.0$ outperforms $\rho=0.0$ by $+3.07\%$ average Micro-F1, with monotonic improvement concentrated on later tasks, where task_C improves by $+5.80\%$ and task_D by $+6.62\%$. Overall, we recommend default values for r , γ , and τ , while ρ should be set to its maximum.

VIII. CONCLUSION

In this paper, we proposed a unified continual learning framework for LLM-based smart contract vulnerability detection that integrates efficient adaptation, forgetting mitigation, and deployment consolidation into a cohesive three-stage pipeline. FA-LoRA achieves competitive detection accuracy with only 0.4% trainable parameters by retaining high-frequency components in the Fourier domain. FAR tracks per-sample loss dynamics to prioritize vulnerable knowledge for rehearsal, attaining the best CL performance among compared methods. APPM consolidates task-specific adapters via anchor-protected merging with frequency-domain gate competition, achieving performance within 2.7% of the independent per-task upper bound at millisecond merge cost. Extensive experiments validate the effectiveness of each component and demonstrate that frequency-domain continual learning provides an effective approach for evolving smart contract security. For future work, 6G space-air-ground integrated

networks form a hierarchical architecture with heterogeneous computation, communication, and storage capabilities, where edge-cloud collaboration and on-device small language models with LLMs offloaded at edge servers introduce asymmetric resource constraints that may affect the adaptation, replay, and merging stages of our framework. Extending FA-LoRA, FAR, and APPM to such heterogeneous environments is a promising direction for future research.

REFERENCES

- [1] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Systematic review of security vulnerabilities in Ethereum blockchain smart contract," *IEEE Access*, vol. 10, pp. 6605–6621, 2022.
- [2] T. Jiao, Z. Xu, M. Qi, S. Wen, Y. Xiang, and G. Nan, "A survey of ethereum smart contract security: Attacks and detection," *Distributed Ledger Technologies: Research and Practice*, vol. 3, no. 3, pp. 1–28, 2024.
- [3] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024.
- [4] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Network and Distributed System Security Symposium*, 2025.
- [5] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, J. Yu, Y. Wang, X. Lin, T. Chen, and Z. Zheng, "When chatgpt meets smart contract vulnerability detection: How far are we?" *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 4, Apr. 2025.

- [6] H. Du, D. Niyato, J. Kang, Z. Xiong, P. Zhang, S. Cui, X. Shen, S. Mao, Z. Han, A. Jamalipour *et al.*, “The age of generative ai and ai-generated everything,” *Ieee Network*, vol. 38, no. 6, pp. 501–512, 2024.
- [7] E. Yang, L. Shen, G. Guo, X. Wang, X. Cao, J. Zhang, and D. Tao, “Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications, and opportunities,” vol. 58, no. 8, Feb. 2026.
- [8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, “LoRA: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.
- [10] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [11] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, “Dark experience for general continual learning: a strong, simple baseline,” *Advances in neural information processing systems*, vol. 33, pp. 15 920–15 930, 2020.
- [12] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.
- [13] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, “Securify: Practical security analysis of smart contracts,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 67–82.
- [14] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “SmartCheck: Static analysis of ethereum smart contracts,” in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2018, pp. 9–16.
- [15] B. Jiang, Y. Liu, and W. K. Chan, “ContractFuzzer: Fuzzing smart contracts for vulnerability detection,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 259–269.
- [16] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, “SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 227–239.
- [17] Z. Liao, Y. Nan, H. Liang, S. Hao, J. Zhai, J. Wu, and Z. Zheng, “SmartAxe: Detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 249–270, 2024.
- [18] Z. Liao, Y. Nan, Z. Gao, H. Liang, S. Hao, J. Wu, and Z. Zheng, “Satellite: Detecting and analyzing smart contract vulnerabilities caused by subcontract misuse,” *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3360–3375, 2025.
- [19] B. Boi, C. Esposito, and S. Lee, “Smart contract vulnerability detection: The role of Large Language Model (LLM),” *SIGAPP Appl. Comput. Rev.*, vol. 24, no. 2, p. 19–29, Aug. 2024.
- [20] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, “Combining fine-tuning and LLM-based agents for intuitive smart contract auditing with justifications,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1742–1754.
- [21] L. Yu, Z. Huang, H. Yuan, S. Cheng, L. Yang, F. Zhang, C. Shen, J. Ma, J. Zhang, J. Lu *et al.*, “Smart-LLaMA-DPO: Reinforced large language model for explainable smart contract vulnerability detection,” vol. 2, no. ISSTA. ACM New York, NY, USA, 2025, pp. 182–205.
- [22] T. Huang, J. Wen, J. Kang, S. Chen, Z. Li, T. Zhang, D. Liu, J. Wang, C. Cai, and Y. Liu, “Paravul: A parallel large language model and retrieval-augmented framework for smart contract vulnerability detection,” *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 5017–5030, 2026.
- [23] Z. Liao, Y. Nan, Z. Gao, H. Liang, S. Hao, P. Ren, and Z. Zheng, “Augmenting smart contract decompiler output through fine-grained dependency analysis and LLM-facilitated semantic recovery,” *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3574–3590, 2025.
- [24] C. Yang, J. Li, S. Lan, Y. Wang, H. Du, C. Gong, X. Yao, D. T. Niyato, and L. Zhu, “Detecting training data for large language models: A survey,” *ACM Comput. Surv.*, vol. 58, no. 9, Feb. 2026.
- [25] N. Ding, X. Lv, Q. Wang, Y. Chen, B. Zhou, Z. Liu, and M. Sun, “Sparse low-rank adaptation of pre-trained language models,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 4133–4145.
- [26] K. Bhardwaj, N. P. Pandey, S. Priyadarshi, V. Ganapathy, S. Kadambi, R. Esteves, S. Borse, P. Whatmough, R. Garrepalli, M. Van Baalen *et al.*, “Sparse high rank adapters,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 13 685–13 715, 2024.
- [27] Z. Gao, Q. Wang, A. Chen, Z. Liu, B. Wu, L. Chen, and J. Li, “Parameter-efficient fine-tuning with discrete Fourier transform,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 21–27 Jul 2024, pp. 14 884–14 901.
- [28] S. Borse, S. Kadambi, N. P. Pandey, K. Bhardwaj, V. Ganapathy, S. Priyadarshi, R. Garrepalli, R. Esteves, M. Hayat, and F. Porikli, “FouRA: Fourier low-rank adaptation,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 71 504–71 539, 2024.
- [29] L. Wang, X. Zhang, H. Su, and J. Zhu, “A comprehensive survey of continual learning: Theory, method and application,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 46, no. 8, pp. 5362–5383, 2024.
- [30] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, “Continual learning of large language models: A comprehensive survey,” *ACM Comput. Surv.*, vol. 58, no. 5, Nov. 2025.
- [31] J. Schwarz, W. Czarnecki, J. Luketina, A. Grabska-Barwinska, Y. W. Teh, R. Pascanu, and R. Hadsell, “Progress & compress: A scalable framework for continual learning,” in *International conference on machine learning*. PMLR, 2018, pp. 4528–4537.
- [32] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, “Memory aware synapses: Learning what (not) to forget,” in *European conference on computer vision*. Springer, 2018, pp. 144–161.
- [33] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continual learning,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [34] Y. Lin, Z. Gao, H. Du, D. Niyato, J. Kang, and X. Liu, “Incentive and dynamic client selection for federated unlearning,” in *Proceedings of the ACM Web Conference*, 2024, pp. 2936–2944.
- [35] H. Zeng, M. Xu, T. Zhou, X. Wu, J. Kang, Z. Cai, and D. Niyato, “One-shot-but-not-degraded federated learning,” in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 11 070–11 079.
- [36] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi, “Editing models with task arithmetic,” in *International Conference on Learning Representations*, 2023.
- [37] P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal, “Ties-merging: Resolving interference when merging models,” *Advances in neural information processing systems*, vol. 36, pp. 7093–7115, 2023.
- [38] L. Yu, B. Yu, H. Yu, F. Huang, and Y. Li, “Language models are super mario: Absorbing abilities from homologous models as a free lunch,” in *Icml*, vol. 2, no. 8, 2024, p. 21.
- [39] X. Jin, X. Ren, D. Preotiuc-Pietro, and P. Cheng, “Dataless knowledge fusion by merging weights of language models,” in *International Conference on Learning Representations*, 2023.
- [40] M. Matena and C. Raffel, “Merging models with Fisher-weighted averaging,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 17 703–17 716.
- [41] X. Zou, M. Shen, C.-S. Bouganis, and Y. Zhao, “Cached multi-lora composition for multi-concept image generation,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 75 638–75 666.
- [42] D. Tang, P. Yadav, Y.-L. Sung, J. Yoon, and M. Bansal, “LoRA merging with SVD: Understanding interference and preserving performance,” in *ICML 2025 Workshop on Reliable and Responsible Foundation Models*, 2025.
- [43] M. Heidari, Y. Medghalchi, M. Khouresha, R. Rezaeian, and I. Hachililoglu, “WaveRA: Wavelet low rank adaptation,” *arXiv preprint arXiv:2506.24092*, 2025.
- [44] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [45] S. J. Alsunaidi, H. Aljamaan, and M. Hammoudeh, “DIVE: A multi-label smart contract vulnerability dataset,” *Scientific Data*, vol. 13, no. 1, p. 664, 2026.
- [46] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.